Ayoub Fourti

+216 55116920 fourtii.ayoub@gmail.com linkedin.com/Ayoub-Fourti github.com/ayoubfourti

Résumé

Étudiant ingénieur dernière année (systèmes embarqués, bac+5) à ESPRIT Tunis. **Robotique & contrôle-commande :** ROS 2, Nav2, SLAM (Cartographer), fusion LiDAR/IMU/caméra, détection embarquée YOLOv8 (Jetson Nano), sim-to-real (Gazebo), AMR industriel. **IHM & supervision :** Qt/QML, FastAPI, React, monitoring temps réel. **Embarqué :** STM32 bare-metal, ESP32, C/C++, Python, Linux.

Formation

ESPRIT, École d'ingénieurs 2022-présent
Cycle préparatoire intégré et cycle ingénieur *Ariana, Tunisie*

Expérience

Cyber Mech Systems [Voir la démo]
Stagiaire ingénieur logiciel — Plateforme robotique *À distance, été 2026*

- Intégré une pile **ROS 2** sur un **AMR industriel** : contrôle temps réel, navigation autonome et supervision de missions — validée sous **Gazebo** avant déploiement réel.
- Développé une IHM de supervision de flotte (FastAPI/React) : monitoring temps réel, télémétrie, contrôle à distance — et conçu l'architecture d'une IHM locale Qt/QML pour AMR (contrôle manuel, gestion de missions, docking, diagnostics).

Projets académiques

Hexapode de recherche et sauvetage | *ROS 2*, *Jetson Nano*, *ESP32*, *Gazebo*, *SLAM*, *Nav2* [Voir la démo]

- Implémenté **SLAM (Cartographer)**, **Nav2** et **Behavior Trees** pour la navigation autonome, avec fusion **LiDAR / IMU / caméra** et détection embarquée **YOLOv8** sur Jetson Nano.
- Développé les **algorithmes de commande et de démarche** pour le contrôle temps réel de **18 actionneurs** sur **ESP32** : estimation d'état, gestion des transitions de phase et contrôle de supervision.
- Validé l'ensemble de la pile sur **Gazebo** (modèle URDF, plugins contacts et capteurs) avant déploiement sur robot réel — identification et correction des écarts **sim-to-real**.

Smart AquaCare — Prototypage & intégration capteurs | *STM32*, *C bare-metal*, *ADC/DMA*, *PWM*, *IoT* [Voir la démo]

- Développé un **contrôleur embarqué temps réel** sur **STM32** en C bare-metal : pilotage d'actionneurs par **PWM et timers**, acquisition capteurs par **ADC + DMA** non bloquante, gestion des événements par **interruptions**.
- Automatisé la planification avec le **RTC** et déployé un **modèle d'IA léger** via **STM32Cube.AI**.

Bracelet de détection de chute (FPGA) | *VHDL*, *FPGA*, *Nios II*, *Quartus*, *ModelSim* [Voir la démo]

- Conçu en **VHDL** une architecture modulaire de détection de chute pour personnes âgées : **machine d'états de Moore**, temporisateur, détection de seuils et compteur, validée par un **testbench de 6 scénarios** sous **ModelSim** (projet en équipe).
- Réalisé la même fonction en **codesign matériel/logiciel** sur **FPGA** : système **Nios II** sous Platform Designer avec logique applicative en **C**.

Wireless Auto — Système IoT de surveillance | *ESP32*, *MQTT*, *Raspberry Pi* [Voir la démo]

- Conçu un réseau de **capteurs ESP32** pour la surveillance en temps réel, reliés par **MQTT** à une **passerelle Raspberry Pi** — architecture proche des solutions IoT de monitoring terrain.
- Mis en place une **supervision à distance** via tableau de bord IoT.

Compétences

Langages : C/C++, Embedded C, Python, VHDL, Java, JavaScript, TypeScript, SQL, Bash
Embarqué & firmware : Microcontrôleurs (STM32, ESP32, Arduino), DMA, Timers, Interruptions, UART, I2C, SPI, Linux embarqué (Yocto, Buildroot)
FPGA & conception numérique : VHDL, Quartus Prime, ModelSim, Platform Designer, Nios II, codesign matériel/logiciel
Robotique & contrôle-commande : ROS 2, vision par ordinateur (YOLOv8, OpenCV), détection embarquée (Jetson Nano), contrôle temps réel d'actionneurs, STM32 bare-metal (PWM, timers, ADC/DMA), ESP32, Gazebo (sim-to-real), Nav2, fusion multicapteurs
IHM & supervision : Qt/QML, FastAPI, React, monitoring temps réel, supervision de flotte, architecture HMI multi-pages
Outils : Git/GitHub, Docker, RViz, YOLOv8, AWS (certifié), Proteus, VS Code
Langues : Français (B2), Anglais (B1)